Creative Software Design
2. Lab2
C: `struct`

Dept. of Computer Science

Hanyang University

Pointers to Constant vs. Constant Pointers

- Pointers to constant

```
const int* ptr = &num;
```

  - The pointers can have new addresses
  - The the value that pointers point to cannot be updated

- Constant pointers

```
int* const ptr = &num;
```

  - The pointers cannot have new addresses

Type `struct`

- *User-defined* and *composite* types that can hold
  - Primitive types
  - Arrays of some types
  - Other struct types

- Ex) A data type representing a "book":

```
struct Book
{
   char  title[50];
   char  author[50];
   char  subject[100];
   int   book_id;
};
```

Name: `Book`

Members: `title[50]`, `author[50]`, `subject[100]`, `book_id`

Using `struct` Variables

- First declaring types

```
struct Book {
  // ... //
  int   book_id;
};
```

- Declaring variables with the types

```
struct Book book1;
```

- Accessing a member with dot(.) operator

structure variable: `book1`

member name: `book_id`

member access operator: `.`

```
book1.book_id = 0;
// Assign 0 to the member
book_id of the structure
variable book1
```

Pointers to `struct` Data

- -> operator simplifies accessing a member of struct in pointers

```
struct point pos={11, 12};
struct point * pptr=&pos;
 // a pointer to struct point
(*pptr).xpos=10;
 // accessing xpos in struct point pointed by pptr
(*pptr).ypos=20;
 // accessing ypos in struct point pointed by pptr
```

```
(*pptr).xpos=10;  ⟺  pptr->xpos=10;

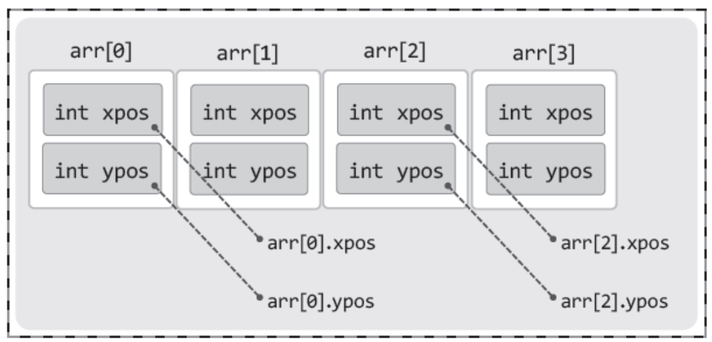
(*pptr).ypos=20;  ⟺  pptr->ypos=20;
```


Arrays of `struct` Data

- Arrays can have any types, which is identical
  - We want to make 4 points as an array

    ➔ Point arr[4];

```
typedef
struct
{
    int xpos;
    int ypos;
} Point;
```



1. Share Values btw *caller* and *callee*

- Common methods for passing data btw caller and callee
  - Passing or returing a value
  - Use references (using pointers)
  - Use a global variable

```
#include <stdio.h>

int main(void) {
  int a = 10, b = 20;

  /* TODO: print the result of the addition of a and b */

  return 0;
}
```


2. Swapping Primitive Values

- Store two integers through **`scanf()`**
- Swap their values
  - Generally, functions cannot return two values at the same time

```
#include <stdio.h>

int main(void) {
  int a, b;
  printf("Input two numbers: ");
  scanf("%d %d", &a, &b);
  printf("Your numbers were %d %d\n", a, b);
  printf("After swaption...\n");
  // TODO:
  printf("Your numbers are %d %d\n", a, b);
  return 0;
}
```

```
Input two numbers: 10 20
Your numbers were 10 and 20
swapping.....
Your numbers are 20 and 10
```

3. Min/Max Values in Arrays

- Fill values in an array of size 5
- Find min and max value in the array using ***one*** function
  - Generally, functions cannot return two values at the same time

```c
#include <stdio.h>

int main(void) {
  int arr[5], min, max;

  printf("Let's fill an array of size 5\n");
  for (int i = 0; i < 5; i++) {
    printf("arr[%d]: ", i);
    scanf("%d", &arr[i]);
  }

  // TODO: complete find_min_max(..);

  printf("Min: %d, Max: %d in the array\n",
      min, max);

  return 0;
}
```

```
Let's fill an array of size 5
arr[0]: 3
arr[1]: 45
arr[2]: 2
arr[3]: 6
arr[4]: -1
Min: -1, Max: 45 in the array
```


4. Bookstore

- You can register 3 books
- Each book has at an least ID, title, and writer property
- Shows your registered books

```
ID for new book: 1
Title: Three_easy_pieces
Writer: Alice
ID for new book: 2
Title: Computer_systems
Writer: Randal
ID for new book: 3
Title: Linux_kernel
Writer: Bovet

-----------------------------------------------------------------
Books[0] => {ID: 1, title: "Three_easy_pieces", writer: "Alice"}
Books[1] => {ID: 2, title: "Computer_systems", writer: "Randal"}
Books[2] => {ID: 3, title: "Linux_kernel", writer: "Bovet"}
```

4. Bookstore (Cnt'd)

- Simply the type `book` by using `typedef`

```
struct point
{
    int xpos;
    int ypos;
};

typedef struct point Point;
```

```
typedef struct point
{
    int xpos;
    int ypos;
} Point;
```

```
typedef struct
{
    int xpos;
    int ypos;
} Point;
```